

Architecture Decision Records (ADR)

Format: Context · Options · Decision · Consequences (trade-offs).

ADR-001: Relational (SQL) database as the inventory source of truth

Status: Accepted

Context: Inventory must never oversell; reservation, inventory and outbox changes must commit atomically.

Options: (A) PostgreSQL/MySQL; (B) NoSQL (DynamoDB/Cassandra/Mongo); (C) Redis only.

	SQL	NoSQL	Redis only
Atomic multi-row transaction	Yes	Limited/costly (conditional writes on one item)	Lua only, durability weak
Constraints (CHECK, UNIQUE)	Yes	Limited	No
Horizontal write scaling	Harder	Easy	Easy
Durability / RPO 0	Sync replication	Tunable	Weak

Decision: SQL (PostgreSQL) for inventory, payment, order. NoSQL/Redis for cart, cache, sessions, read models.

Consequences: Strong correctness and audit; the hot row limits write throughput → mitigated by funnel (only ~100 writes) and stock bucketing. Separate DB per service.

ADR-002: Concurrency control - atomic conditional UPDATE + Redis gate (not pure pessimistic/optimistic)

Status: Accepted

Options: Pessimistic `SELECT FOR UPDATE`; Optimistic version CAS with retry; Atomic conditional UPDATE; Single-writer queue per SKU.

Decision: Atomic `UPDATE ... WHERE available_quantity >= q` in one transaction with reservation insert; Redis Lua counter as a pre-filter; `CHECK` constraints as final guard. Optimistic `version` used for low-contention updates.

Consequences: Short critical section, no retry storms, DB protected from 9,900 losers. Cost: Redis counter can drift → reconciliation; extra component. Fallback to single-writer queue / bucketing if one SKU outgrows a row.

ADR-003: Admission control / virtual waiting room in front of inventory

Context: 10,000 (up to 500k req/s) at once.

Options: Let everything through; rate limit only; waiting room + token bucket.

Decision: Waiting room with token bucket and randomised ordering after sale start.

Consequences: Bounded downstream load, predictable latency, fair-ish. Users may wait; extra component; true FIFO fairness is not guaranteed.

ADR-004: Temporary reservation with TTL (reserve → pay → confirm)

Options: Decrement stock only on payment success (oversell risk / many failed payments); decrement at reserve with TTL.

Decision: Reserve at Buy Now, TTL 10 min (configurable), release on failure/timeout; confirmed reservations are exempt from expiry.

Consequences: No oversell and fair holding; stock may be temporarily locked by non-paying users (mitigated by short TTL, per-user limit, waitlist re-offer). Requires sweeper and DB-time expiry.

ADR-005: Synchronous reserve, asynchronous post-payment pipeline

Operation	Mode	Why
Reserve	Sync	Customer needs an immediate deterministic result; single atomic op
Payment initiation	Sync-short → returns PENDING	Needs redirect/token; final result async
Payment → Order → Shipment → Notification	Async (events)	Decoupling, retry, buffering during Order outage

Consequences: Resilience and scale; eventual consistency between payment and order (seconds) and harder debugging (mitigated by tracing).

ADR-006: Saga (orchestrated) + Transactional Outbox instead of 2PC

Options: Distributed 2PC/XA; choreography only; orchestrated saga + outbox.

Decision: CheckoutSaga orchestrates reserve/pay; services publish via outbox; consumers idempotent; reconciliation as safety net.

Consequences: No distributed locks and no cross-service coupling at DB level; must design compensations (refund, release) and accept eventual consistency; at-least-once means duplicates → idempotency everywhere.

ADR-007: Idempotency at every mutation boundary

Decision: Client Idempotency-Key + DB unique constraints (reservation , payment , order , notification) + gateway merchant_ref + processed_event for consumers.

Consequences: Duplicate clicks, retries, redeliveries and late webhooks are harmless; cost is dedupe storage and key lifecycle management.

ADR-008: Payment timeout = unknown state, resolved by reconciliation (no blind retry)

Decision: On timeout mark TIMED_OUT , reuse same merchant_ref for any retry, poll/query gateway; never create a new charge reference. Circuit breaker for outages.

Consequences: No double charge; customer sees "processing" longer; requires reconciler and gateway query API.

ADR-009: Kafka (log-based broker) over simple queue

Options: RabbitMQ/SQS; Kafka.

Decision: Kafka: durable retained log, replay, per-key ordering, high throughput, consumer lag autoscaling.

Consequences: Operational complexity; at-least-once semantics; partition planning needed. Managed Kafka recommended.

ADR-010: Cache strategy

Data	Strategy
Static sale/product pages	CDN, TTL 1-60 s, stale-while-revalidate
Stock banner	Derived read model, 1-2 s TTL, updated by events; never used for decisions
Stock gate counter	Redis, authoritative only for fast rejection; rebuilt from DB
Cart	Redis with TTL, snapshot on checkout
Idempotency fast check	Redis, DB constraint is the authority

Consequences: Large load reduction; staleness accepted for display only.

ADR-011: Microservices with database-per-service (bounded contexts)

Decision: Inventory, Payment, Order, Checkout, Shipment, Notification, Product separate; Inventory separate from Order due to different consistency/scaling profile.

Consequences: Independent scaling and failure isolation; more infra and distributed-data complexity (mitigated by events and reconciliation). A modular monolith is acceptable for a small team but would couple scaling of hot inventory to everything else.

ADR-012: Single-region active, multi-AZ with warm DR region

Decision: Multi-AZ synchronous DB standby (RPO 0), warm standby region with async replication.

Consequences: Simpler consistency than multi-region active-active inventory (which would require global coordination); region loss means RTO ~30 min and small data loss risk for async-replicated data only.

ADR-013: Refund-by-default for late payment after reservation release

Context: Payment may succeed after reservation expired.

Decision: If stock cannot be re-reserved atomically, auto-refund and notify; if the unit is still free, re-reserve atomically and confirm.

Consequences: Never oversell; occasional customer disappointment; refund latency.

Summary Table

ADR	Decision	Sacrifices
001	SQL for inventory	Write scalability on hot row
002	Atomic UPDATE + Redis gate	Extra component, drift reconciliation
003	Waiting room	User wait, exact FIFO
004	TTL reservation	Temporary stock lock-up
005	Sync reserve / async downstream	Eventual consistency
006	Saga + outbox	Compensation complexity
007	Idempotency everywhere	Storage, key management
008	Reconcile on timeout	Slower final status
009	Kafka	Ops complexity
010	Layered caching	Stale display data
011	Microservices	Distributed complexity
012	Single-region active	Region-failure RTO
013	Refund on late payment	Customer experience